Data Manipulation and Analysis Using Pandas

Pandas is one of the most important Python libraries for **data manipulation and data analysis**.

The name Pandas comes from **Panel Data**.

Pandas is particularly useful for working with:

- CSV files
- Excel files
- SQL data
- JSON data
- tables
- structured datasets
- missing values
- duplicate records
- categorical data
- dates and times

- **Why Pandas?**
- Suppose we have this student dataset:

Student	Age	Gender	Marks	Department
Rahul	20	M	85	CSE
Priya	21	F	92	CSE
Anil	20	M	67	ECE
Meena	22	F	78	CSE

If we want to:

- find average marks
- select CSE students
- find students scoring above 80
- remove duplicates
- handle missing values
- sort students by marks
- create a new column
- group students by department

Pandas makes these operations very easy.

Importing Pandas

The standard convention is:

import pandas as pd

Here:

pd

is an alias for Pandas.

We can then write:

pd.DataFrame()

pd.read_csv()

pd.read_excel()

instead of:

pandas.DataFrame()

pandas.read_csv()

pandas.read_excel()

Important Pandas Data Structures

There are two fundamental Pandas data structures:

- **Series**
- **DataFrame**

Pandas Series

- A **Series** is a one-dimensional labelled data structure.

Example:

```
import pandas as pd
marks = pd.Series([75, 80, 65, 90, 85])
print(marks)
```

Output:

```
0    75
1    80
2    65
3    90
4    85
dtype: int64
```

Notice that Pandas automatically creates an index:

Creating a Series with Custom Index

```
marks = pd.Series(
    [85, 92, 67],
    index=["Rahul", "Priya", "Anil"]
)

print(marks)
```

Output:

```
Rahul    85
Priya    92
Anil     67
dtype: int64
```

Now we can access:

```
print(marks["Priya"])
```

Output:

92

A **DataFrame** is a two-dimensional labelled data structure.

You can think of it as a table in Excel, a database, a CSV file

Example:

```
import pandas as pd

data = {
    "Name": ["Rahul", "Priya", "Anil", "Meena"],
    "Age": [20, 21, 20, 22],
    "Marks": [85, 92, 67, 78]
}

df = pd.DataFrame(data)

print(df)
```

Output:

```
    Name  Age  Marks
0  Rahul   20     85
1  Priya   21     92
2   Anil   20     67
3  Meena   22     78
```

Pandas automatically creates:

0

1

2

So a DataFrame consists of:

Rows + Columns + Index

Viewing Data

head()

Displays the first five rows.

print(df.head())

We can specify the number:

print(df.head(2))

tail()

Displays the last five rows.

```
print(df.tail())
```

Or:

```
print(df.tail(2))
```

Understanding Dataset Structure

shape

print(df.shape)

Example: (4, 3)

This means: 4 rows 3 columns

columns

print(df.columns)

Output:

Index(['Name', 'Age', 'Marks'], dtype='object')

index

print(df.index)

dtypes

print(df.dtypes)

Example:

```
Name     object
Age       int64
Marks     int64
dtype: object
```

info()

One of the most useful functions for understanding a dataset is:

df.info()

It provides:

number of rows

column names

number of non-null values

data types

memory usage

Example output:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 4 entries, 0 to 3

Data columns (total 3 columns):

Name 4 non-null object

Age 4 non-null int64

Marks 4 non-null int64

Statistical Summary — describe()

print(df.describe())

For numerical columns, Pandas provides:

Count mean standard deviation

Minimum 25th percentile median 75th percentile maximum

Example:

	Age	Marks
count	4.000000	4.000000
mean	20.750000	80.500000
std	0.957427	10.408330
min	20.000000	67.000000
25%	20.000000	75.250000
50%	20.500000	81.500000
75%	21.250000	86.750000
max	22.000000	92.000000

This is extremely useful in exploratory data analysis.

IMPORTING DATASETS

Why Import External Data?

Real-world data is rarely typed manually into Python.

It may be stored in:

CSV

Excel

JSON

database

text files

Pandas provides functions for importing these datasets.

Importing a CSV File

Suppose we have:

students.csv

We can load it using:

```
import pandas as pd

df = pd.read_csv("students.csv")

print(df)
```

Example CSV Dataset

Suppose students.csv contains:

Name,Age,Department,Marks

Rahul,20,CSE,85

Priya,21,CSE,92

Anil,20,ECE,67

Meena,22,CSE,78

Kiran,21,ISE,88

Reading an Excel File

For an Excel file:

```
df = pd.read_excel("students.xlsx")
```

Reading a Specific Excel Sheet

```
df = pd.read_excel(
    "students.xlsx",
    sheet_name="Students"
)
```

EXPORTING DATASETS

Export DataFrame to CSV

Suppose:

```
df = pd.DataFrame({
    "Name": ["Rahul", "Priya"],
    "Marks": [85, 92]
})
```

Export:

```
df.to_csv("students_output.csv", index=False)
```

Why index=False?

Without it, Pandas may export its index as an additional column.

Export DataFrame to Excel

```
df.to_excel(
    "students_output.xlsx",
    index=False
)
```

```
import pandas as pd
df = pd.read_csv("students.csv")
df["Marks"] = df["Marks"] + 5
df.to_csv(
    "updated_students.csv",
    index=False
)
```

Loading JSON data into a DataFrame

Create a JSON file named students.json

```
[
   {
      "RollNo": 101,
      "Name": "Anil",
      "Marks": 85,
      "City": "Mangalore"
   },
   {
      "RollNo": 102,
      "Name": "Priya",
      "Marks": 92,
      "City": "Bangalore"
   },
   {
      "RollNo": 103,
      "Name": "Rahul",
      "Marks": 78,
      "City": "Mysore"
   },
   {
      "RollNo": 104,
      "Name": "Sneha",
      "Marks": 88,
      "City": "Udupi"
   }
]
```

```
import pandas as pd

# Load JSON data into a DataFrame
df = pd.read_json("students.json")

# Display the DataFrame
print(df)

# Display the first two records
print("\nFirst two records:")
print(df.head(2))

# Display DataFrame information
print("\nDataFrame information:")
print(df.info())
```

Writing a DataFrame to a JSON file

```
import pandas as pd
# Create a DataFrame
data = {
    "RollNo": [101, 102, 103],
    "Name": ["Anil", "Priya", "Rahul"],
    "Marks": [85, 92, 78]
}

df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)
# Write the DataFrame to a JSON file
df.to_json("students_output.json", orient="records", indent=4)

print("\nDataFrame successfully written to JSON file.")
```

orient="records"

Stores each DataFrame row as a separate JSON object

```
import pandas as pd
import json

# Step 1: Create sample JSON data
students = [
    {"RollNo": 101, "Name": "Anil", "Marks": 85},
    {"RollNo": 102, "Name": "Priya", "Marks": 92},
    {"RollNo": 103, "Name": "Rahul", "Marks": 78}
]

# Step 2: Save sample data to a JSON file
with open("students.json", "w") as file:
    json.dump(students, file, indent=4)

# Step 3: Load JSON into a DataFrame
df = pd.read_json("students.json")

print("Data loaded from JSON:")
print(df)
```

```
# Step 4: Add a new column
df["Result"] = df["Marks"].apply(
    lambda mark: "Pass" if mark >= 40 else "Fail"
)

print("\nUpdated DataFrame:")
print(df)

# Step 5: Write DataFrame to a new JSON file
df.to_json(
    "students_updated.json",
    orient="records",
    indent=4
)

print("\nUpdated data saved to students_updated.json")
```

```
# Step 6: Read back the output JSON file
df_check =
pd.read_json("students_updated.json")

print("\nVerify saved data:")
print(df_check)
```

SELECTING DATA

Selecting a Column

Suppose:

```
df = pd.DataFrame({
    "Name": ["Rahul", "Priya", "Anil"],
    "Age": [20, 21, 20],
    "Marks": [85, 92, 67]
})
```

Select Marks:

```
print(df["Marks"])
```

Output:

```
0    85
1    92
2    67
```

Selecting Multiple Columns

```
print(df[["Name", "Marks"]])
```

Output:

```
    Name  Marks
0  Rahul     85
1  Priya     92
2   Anil     67
```

Important distinction

One column:

```
df["Marks"]
```

Multiple columns:

```
df[["Name", "Marks"]]
```

Selecting Rows with iloc

iloc uses integer positions.

print(df.iloc[0])

First row.

print(df.iloc[1])

Second row.

Selecting Multiple Rows

print(df.iloc[0:2])

Selecting a Specific Cell

print(df.iloc[1, 2])

This means:

row 1

column 2

Selecting Using loc

loc is label-based.

```
print(df.loc[0])
```

We can also select specific columns:

```
print(df.loc[0, "Marks"])
```

Output:

85

iloc vs loc

Method Selection

iloc	Integer position
loc	Label

Example:

df.iloc[0, 2]

means first row, third column.

Whereas:

df.loc[0, "Marks"]

means row label 0, column named Marks.

FILTERING DATA

Students with Marks Greater Than 80

```
result = df[df["Marks"] > 80]

print(result)
```

Students Who Passed

```
passed = df[df["Marks"] >= 40]
print(passed)
```

Students with Marks Between 60 and 80

```
result = df[
    (df["Marks"] >= 60) &
    (df["Marks"] <= 80)
]

print(result)
```

Multiple Conditions

Students from CSE with marks above 80:

```
result = df[
    (df["Department"] == "CSE") &
    (df["Marks"] > 80)
]

print(result)
```

Important

For Pandas:

AND &

OR |

NOT ~

Parentheses should be used around each condition.

OR Condition

Students from CSE or ECE:

```
result = df[
    (df["Department"] == "CSE") |
    (df["Department"] == "ECE")
]
print(result)
```

. isin()

Instead of writing multiple OR conditions:

```
result = df[
    df["Department"].isin(["CSE", "ECE"])
]
```

This is cleaner and easier to read.

SORTING

Sorting by Marks

```
df_sorted = df.sort_values("Marks")
print(df_sorted)
```

Ascending order is the default.

Descending Order

```
df_sorted = df.sort_values(
    "Marks",
    ascending=False
)

print(df_sorted)
```

Sorting by Multiple Columns

```
df.sort_values(
    ["Department", "Marks"],
    ascending=[True, False]
)
```

This means:

Sort Department alphabetically.

Within each department, sort Marks from highest to lowest.

ADDING AND MODIFYING COLUMNS

Creating a New Column

Suppose:

df["Marks"]

contains marks.

We can create a grade:

df["Grade"] = "F“

Now every row initially has F.

Creating Grade Using np.where()

```
import numpy as np

df["Result"] = np.where(
    df["Marks"] >= 40,
    "Pass",
    "Fail"
)
```

. Creating Grade Using apply()

```
def calculate_grade(mark):
    if mark >= 90:
        return "A+"
    elif mark >= 80:
        return "A"
    elif mark >= 70:
        return "B"
    elif mark >= 60:
        return "C"
    elif mark >= 40:
        return "D"
    else:
        return "F“
```

Apply the function:

```
df["Grade"] = df["Marks"].apply(calculate_grade)
```

Creating a Total Column

Suppose we have:

```
df = pd.DataFrame({
    "Name": ["Rahul", "Priya", "Anil"],
    "Internal": [25, 28, 20],
    "External": [60, 65, 45]
})
```

Create total:

```
df["Total"] = (
    df["Internal"] +
    df["External"]
)
```

DATA CLEANING

Real-world datasets are rarely clean.

Typical problems include:

- missing values
- duplicate records
- extra spaces
- inconsistent capitalization
- incorrect data types
- spelling variations
- invalid values
- inconsistent dates

The data-cleaning process is therefore extremely important.

Detecting Missing Values

Consider:

```
df = pd.DataFrame({
    "Name": ["Rahul", "Priya", "Anil", "Meena"],
    "Age": [20, 21, None, 22],
    "Marks": [85, None, 67, 78]
})
```

Check missing values:

```
print(df.isnull())
```

Output will contain True where values are missing.

Count Missing Values

```
print(df.isnull().sum())
```

Example:

```
Name     0
Age      1
Marks    1
dtype: int64
```

This is one of the most useful commands when inspecting a dataset.

Find Rows Containing Missing Values

```
print(df[df.isnull().any(axis=1)])
```

This displays rows containing at least one missing value.

Removing Missing Values

Use:

df.dropna()

This removes rows containing missing values.

Example:

cleaned = df.dropna()

print(cleaned)

Important

dropna() does not modify the original DataFrame unless assigned back or used with appropriate parameters.

df = df.dropna()

Filling Missing Values

Instead of deleting records, we can fill missing values.

df["Marks"] = df["Marks"].fillna(0)

Now missing Marks become 0.

Filling with Mean

A better approach for some numerical datasets is to use the mean.

```
mean_marks = df["Marks"].mean()

df["Marks"] = df["Marks"].fillna(mean_marks)
```

This is an important data-cleaning technique.

Filling Missing Age

```
df["Age"] = df["Age"].fillna(
    df["Age"].mean()
)
```

Detecting Duplicates

print(df.duplicated())

This returns True for duplicate rows.

Count Duplicates

print(df.duplicated().sum())

.Remove Duplicates

df = df.drop_duplicates()

Duplicate Based on a Particular Column

Suppose duplicate student IDs are not allowed:

```
df = df.drop_duplicates(
    subset=["Student_ID"]
)
```

This is useful in real datasets.

STRING CLEANING

Removing Extra Spaces

Suppose:

" Rahul "

We want:

"Rahul"

Use:

df["Name"] = df["Name"].str.strip()

Convert to Uppercase

```
df["Name"] = df["Name"].str.upper()
```

Convert to Lowercase

```
df["Name"] = df["Name"].str.lower()
```

Proper Capitalization

```
df["Name"] = df["Name"].str.title()
```

Replacing Values

Suppose: M Male male

all represent the same category.

We can standardize them:

```
df["Gender"] = df["Gender"].replace({
    "M": "Male",
    "male": "Male"
})
```

DATE/TIME CLEANING

Dates are very common in business datasets.

Suppose:

```
df["Date"] = [
    "2026-01-15",
    "2026-02-20",
    "2026-03-10"
]
```

Convert to datetime:

```
df["Date"] = pd.to_datetime(df["Date"])
```

Now Pandas recognizes these as dates.

Extract Year

```
df["Year"] = df["Date"].dt.year
```

Extract Month

```
df["Month"] = df["Date"].dt.month
```

Extract Day

```
df["Day"] = df["Date"].dt.day
```

Extract Day Name

```
df["Day_Name"] = df["Date"].dt.day_name()
```

GROUPING AND AGGREGATION

Grouping by Department

Consider:

```
df = pd.DataFrame({
    "Name": ["Rahul", "Priya", "Anil", "Meena", "Kiran"],
    "Department": ["CSE", "CSE", "ECE", "CSE", "ECE"],
    "Marks": [85, 92, 67, 78, 88]
})
```

Find average marks by department:

```
result = df.groupby("Department")["Marks"].mean()

print(result)
```

Output:

```
Department
CSE    85.0
ECE    77.5
```

Total Marks by Department

```
df.groupby("Department")["Marks"].sum()
```

Maximum Marks by Department

```
df.groupby("Department")["Marks"].max()
```

Multiple Aggregations

```
result = df.groupby("Department")["Marks"].agg(
    ["mean", "max", "min", "count"]
)
print(result)
```

Rename Columns

Suppose:

df.columns

contains:

Name

Age

Marks

We can rename:

```
df = df.rename(
    columns={
        "Name": "Student_Name",
        "Marks": "Total_Marks"
    }
)
```

Delete a Column

```
df = df.drop(
    columns=["Age"]
)
```

Multiple columns:

```
df = df.drop(
    columns=["Age", "Department"]
)
```

RESETTING INDEX

After filtering or deleting rows, the index might look like:

0

2

5

8

We can reset it:

df = df.reset_index(drop=True)

Now it becomes:

0

1

2

3

MERGING DATASETS

Suppose we have two DataFrames.

Student information

```
students = pd.DataFrame({
    "ID": [101, 102, 103],
    "Name": ["Rahul", "Priya", "Anil"]
})
```

Marks information

```
marks = pd.DataFrame({
    "ID": [101, 102, 103],
    "Marks": [85, 92, 67]
})
```

Merge them:

```
result = pd.merge(
    students,
    marks,
    on="ID"
)
print(result)
```

CONCATENATING DATA

Suppose we have:

```
df1 = pd.DataFrame({
    "Name": ["Rahul", "Priya"],
    "Marks": [85, 92]
})

df2 = pd.DataFrame({
    "Name": ["Anil", "Meena"],
    "Marks": [67, 78]
})
```

Combine them:

```
result = pd.concat(
    [df1, df2],
    ignore_index=True
)
print(result)
```

PIVOT TABLE

Pandas can create pivot tables similar to Excel.

Suppose:

```
sales = pd.DataFrame({
    "Region": ["North", "North", "South", "South"],
    "Product": ["Laptop", "Mobile", "Laptop", "Mobile"],
    "Sales": [50000, 30000, 45000, 35000]
})
```

Create a pivot table:

```
pivot = pd.pivot_table(
    sales,
    values="Sales",
    index="Region",
    columns="Product",
    aggfunc="sum"
)

print(pivot)
```

- REFER UNIT9PROGRAM.TXT

Thank You..